Aula 5

- Sub-rotinas: chamada e retorno
- Caraterização das sub-rotinas na perspetiva do "chamador" e do "chamado"
- Convenções adotadas na arquitetura MIPS quanto à:
  - passagem de parâmetros para sub-rotinas
  - devolução de valores de sub-rotinas
  - utilização e salvaguarda de registos
- A *stack* - *c*onceito e operações básicas
- Utilização da *stack* na arquitetura MIPS

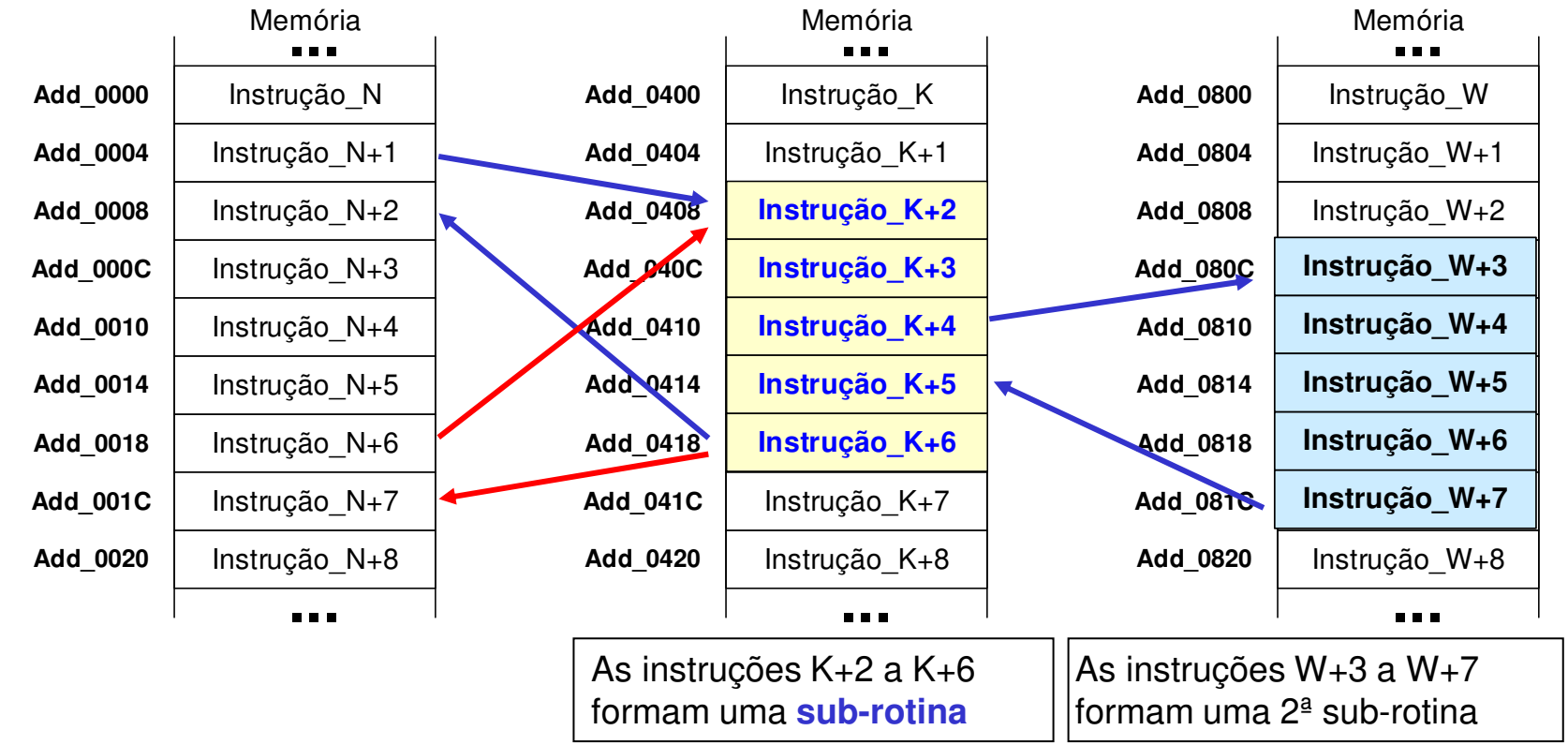
Bernardo Cunha, José Luís Azevedo

Porque se usam funções (sub-rotinas)?

- Há três razões principais que justificam a existência de funções*:
  - A **reutilização no contexto de um determinado programa** - aumento da eficiência na dimensão do código, substituindo a repetição de um mesmo trecho de código por um único trecho evocável de múltiplos pontos do programa
  - A **reutilização no contexto de um conjunto de programas**, permitindo que o mesmo código possa ser reaproveitado (bibliotecas de funções)
  - A **organização e estruturação do código**

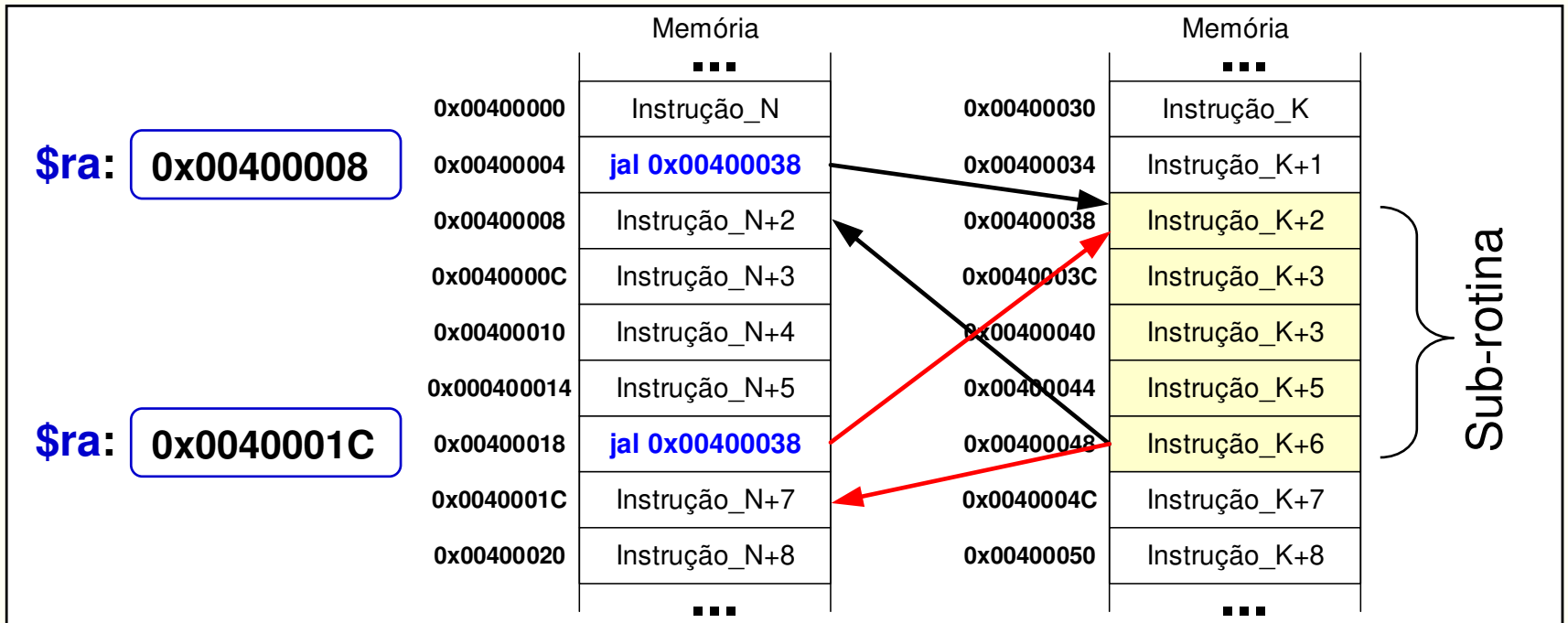
(*) No contexto da linguagem *Assembly*, as funções e os procedimentos são genericamente conhecidas por **sub-rotinas**!

Sub-rotinas: exemplo



- Como implementar o esquema de chamada e retorno de uma sub-rotina?
  - A instrução que chama a sub-rotina tem que guardar o endereço da instrução para onde deve ser feito o retorno ("return address")

Chamada de uma sub-rotina: a instrução JAL



- A ligação (*link*) entre o chamador e o chamado (sub-rotina) é feita pela instrução **JAL (jump and link)**
- A instrução JAL é uma instrução de salto incondicional, que **armazena o valor atual do Program Counter no registo $ra**
- A instrução JAL é codificada do mesmo modo que a instrução J

Retorno de uma sub-rotina: a instrução JR

- Como **regressar** à instrução que sucede à instrução "**jal**" ?
- Aproveita-se o endereço de retorno armazenado em **$ra** durante a execução da instrução "**jal**", usando a instrução "**jr**" ("**jr *register***")

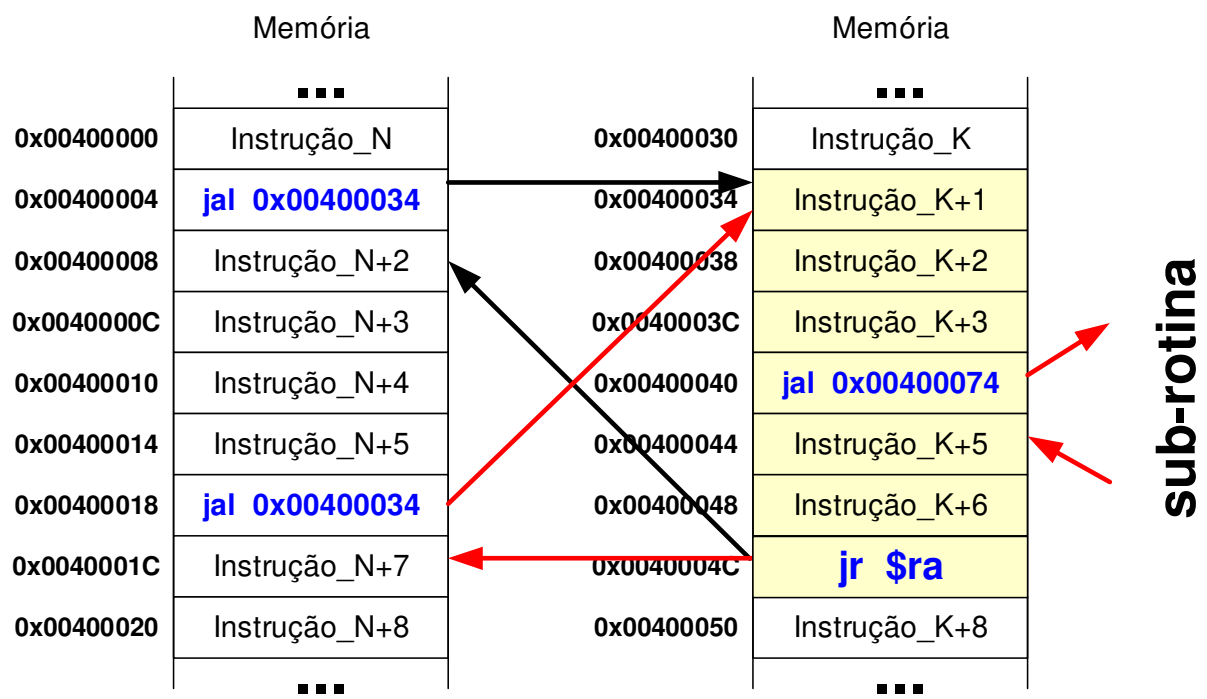
- Programa chamador

```
...
jal  sub1  # guarda o endereço
           # de retorno em $ra
           # e salta para "sub1"
...
```

- Sub-rotina

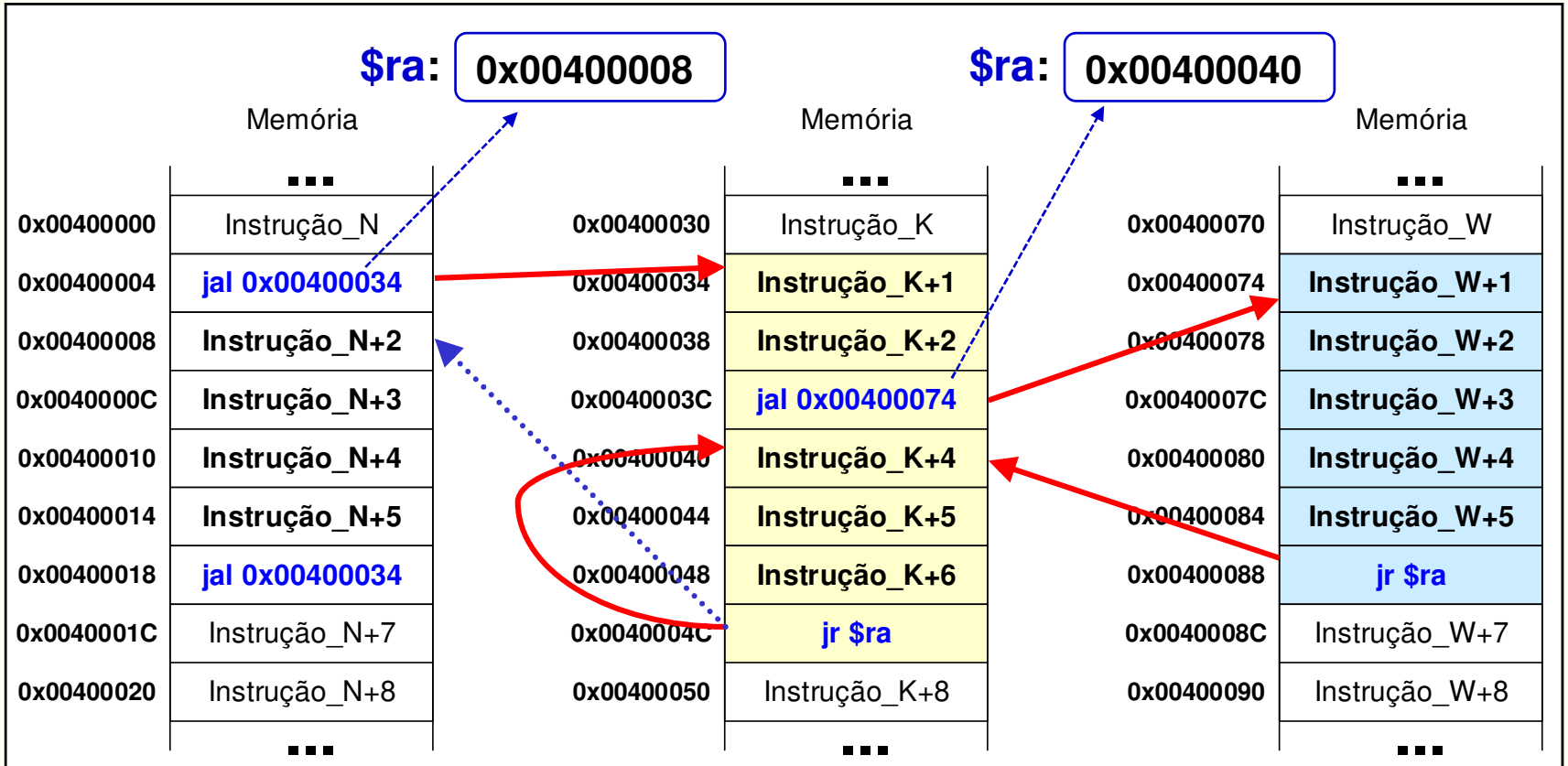
```
sub1:  ...
       ...
       jr   $ra  # regressa ao
                 # programa
                 # chamador
```

Chamada a uma sub-rotina a partir de outra sub-rotina



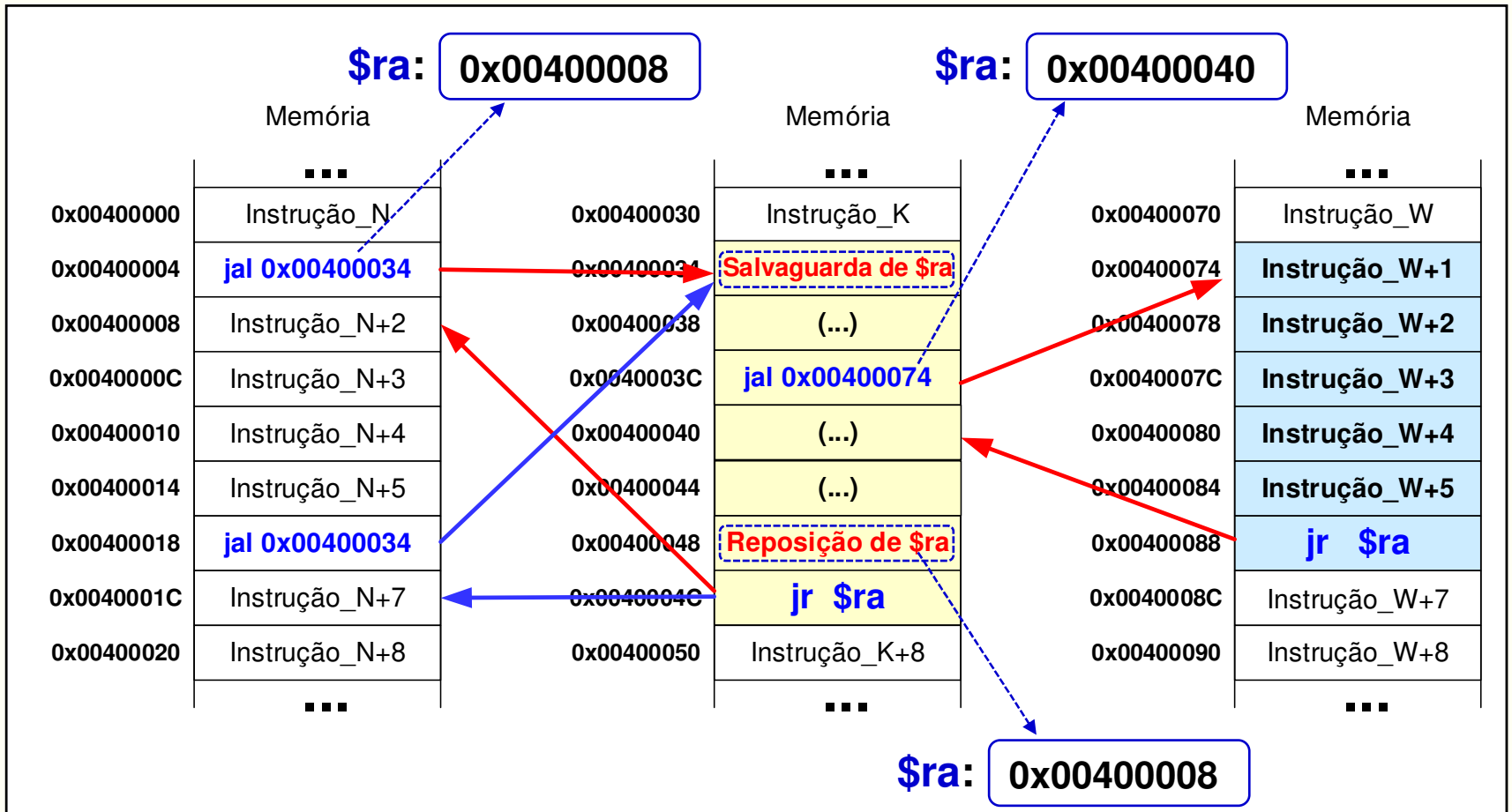
E se a sub-rotina (Instrução_K+1 a “jr $ra”) chamar uma 2ª sub-rotina?

Chamada a uma sub-rotina a partir de outra sub-rotina



- No caso em que a sub-rotina chama uma 2ª sub-rotina, o valor do registo **$ra** é alterado (pela instrução "**jal**"), perdendo-se a ligação para o primeiro chamador. **Como resolver este problema?**

Chamada a uma sub-rotina a partir de outra sub-rotina



- O valor de $ra é salvaguardado, na memória, no início da sub-rotina e é reposto antes do retorno

Instruções JAL e JALR

- A instrução "**`jal`**" é codificada do mesmo modo que a instrução "**`j`**": formato j em que os 26 bits menos significativos são obtidos dos 28 bits menos significativos do endereço-alvo, deslocados à direita dois bits
- Durante a execução, a obtenção do endereço-alvo é feita do mesmo modo que na instrução "**`j`**"
- A especificação de um endereço-alvo de 32 bits é possível através da utilização da instrução "**`jalr`**" (**`jump and link register`**)
- A instrução "**`jalr`**" funciona de modo idêntico à instrução "**`jal`**", exceto na obtenção do endereço-alvo:
  - o endereço da sub-rotina é lido do registo especificado na instrução (endereçamento indireto por registo); ex: **`jalr $t2`**
- A instrução "**`jalr`**" é codificada com o formato R

Utilização de sub-rotinas

- Sub-rotinas são blocos de código com funcionalidade bem definida e interface própria
- As sub-rotinas podem ser reutilizadas sem necessidade de conhecer a sua implementação interna
- As sub-rotinas são frequentemente organizadas em bibliotecas, acessíveis por qualquer programa
- A reutilização exige regras claras de interface:
  - Qual a **funcionalidade** implementada
  - Quais os **parâmetros de entrada** e quais os **resultados de saída** e como são passados
  - Como é preservada a **integridade dos dados** entre programa chamador e sub-rotina (de modo a que um não destrua os dados do outro)

Regras a definir entre chamador e a sub-rotina chamada

- Ao nível da interface:
  - Como **passar parâmetros** do "chamador" para a sub-rotina, identificar quantos e onde são passados
  - Como **devolver**, para o "chamador", resultados produzidos pela sub-rotina
- Ao nível das regras que preservam a integridade dos dados:
  - Que registos do CPU podem "chamador" e sub-rotina usar, sem que haja alteração indevida de informação por parte da sub-rotina (por exemplo a sub-rotina alterar o conteúdo de um registo que é usado pelo chamador e que tem informação que não pode ser perdida)
  - Como partilhar a memória usada para armazenar dados, sem risco de sobreposição (e consequente perda de informação armazenada)

Convenções do MIPS (passagem e devolução de valores)

- Os parâmetros que possam ser armazenados na dimensão de um registo (32 bits, i.e., **char**, **int**, **ponteiros**) devem ser passados à sub-rotina nos registos **`$a0` a `$a3`** ($4 a $7) por esta ordem
  - o **primeiro parâmetro sempre em `$a0`**, o **segundo em `$a1`** e assim sucessivamente
- *Caso o número de parâmetros a passar seja superior a quatro, os 4 primeiros são passados nos registos $ai e os restantes (pela ordem em que são declarados) deverão ser passados na stack*
- A sub-rotina pode devolver um valor de 32 bits ou um de 64 bits:
  - Se o valor a devolver é de **32 bits** é utilizado o registo **`$v0`**
  - Se o valor a devolver é de **64 bits**, são utilizados os registos **`$v1` (32 bits mais significativos) e `$v0` (32 bits menos significativos**)

Preservação da integridade dos dados

- O uso de registos pela sub-rotina pode destruir informação vital do programa chamador
- Que registos pode usar uma sub-rotina?
- **Hipótese 1**: Dividir, de forma estática, os registos existentes entre “chamador” e “chamado”!
  - Nesse caso, o que fazer quando o “chamado” é simultaneamente "chamador" (sub-rotina que chama outra sub-rotina)?
- **Hipótese 2**: **Salvaguardar registos** - copiar para memória antes de usar e repor no final
  - A responsabilidade da salvaguarda pode ser do chamador, da sub-rotina ou dos dois
  - Garante funcionamento correto mesmo quando uma sub-rotina também atua como chamador

Estratégias para a salvaguarda de registos

- Estratégia "**caller-saved**"
  - O programa "chamador" é responsável por salvaguardar o conteúdo dos registos antes de chamar a sub-rotina
  - Cabe-lhe também a tarefa de repor posteriormente o seu valor
  - É possível salvaguardar apenas os registos cujo conteúdo venha a precisar mais tarde
  - **Desvantagem:** aumenta o tamanho do código, pois obriga a incluir instruções de salvaguarda e reposição em todas as chamadas a sub-rotinas
- Estratégia "**callee-saved**"
  - A sub-rotina é responsável por salvaguardar os registos que vai usar
  - Deve restaurar os registos antes de retornar ao chamador
  - **Vantagem:** o código de salvaguarda e reposição fica concentrado no início e no fim da sub-rotina, evitando repetição em cada chamada

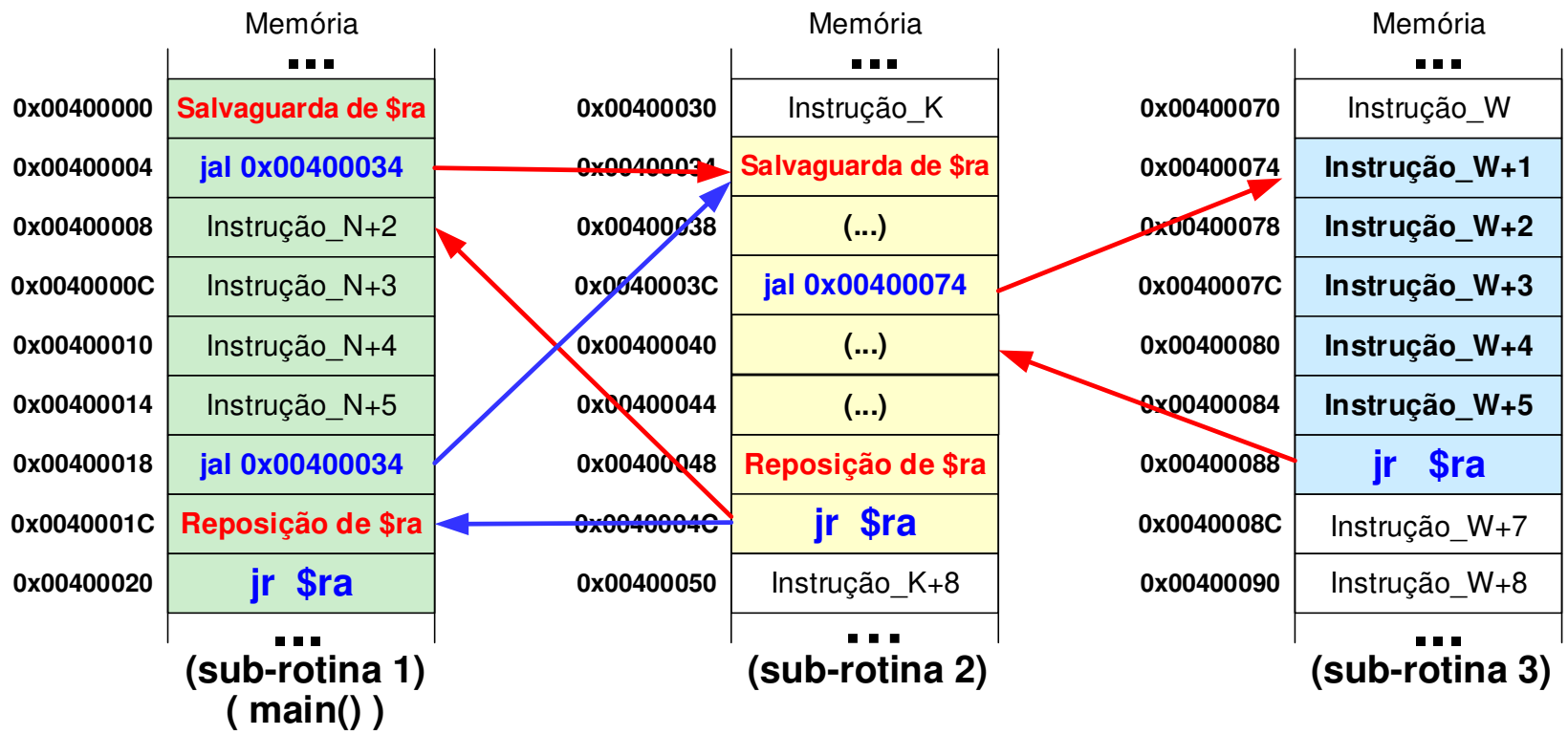
Convenção para salvaguarda de registos no MIPS

- Os registos **`$t0`** a **`$t9`**, **`$v0`** e **`$v1`**, e **`$a0`** a **`$a3`** podem ser livremente utilizados e alterados pelas sub-rotinas
- Os registos **`$s0`** a **`$s7`** não podem, **na perspetiva do chamador**, ser alterados pelas sub-rotinas
  - Se uma dada sub-rotina precisar de usar qualquer um dos registos **`$s0`** a **`$s7`** compete a essa sub-rotina **salvaguardar previamente o seu conteúdo**, repondo-o imediatamente antes de terminar (estratégia "callee-saved")
  - Ou seja, é seguro para o programa chamador usar um registo **`$sn`** para armazenar um valor que vai necessitar após a chamada à sub-rotina, uma vez que tem a garantia que esta não o modifica

Considerações práticas sobre a utilização da convenção

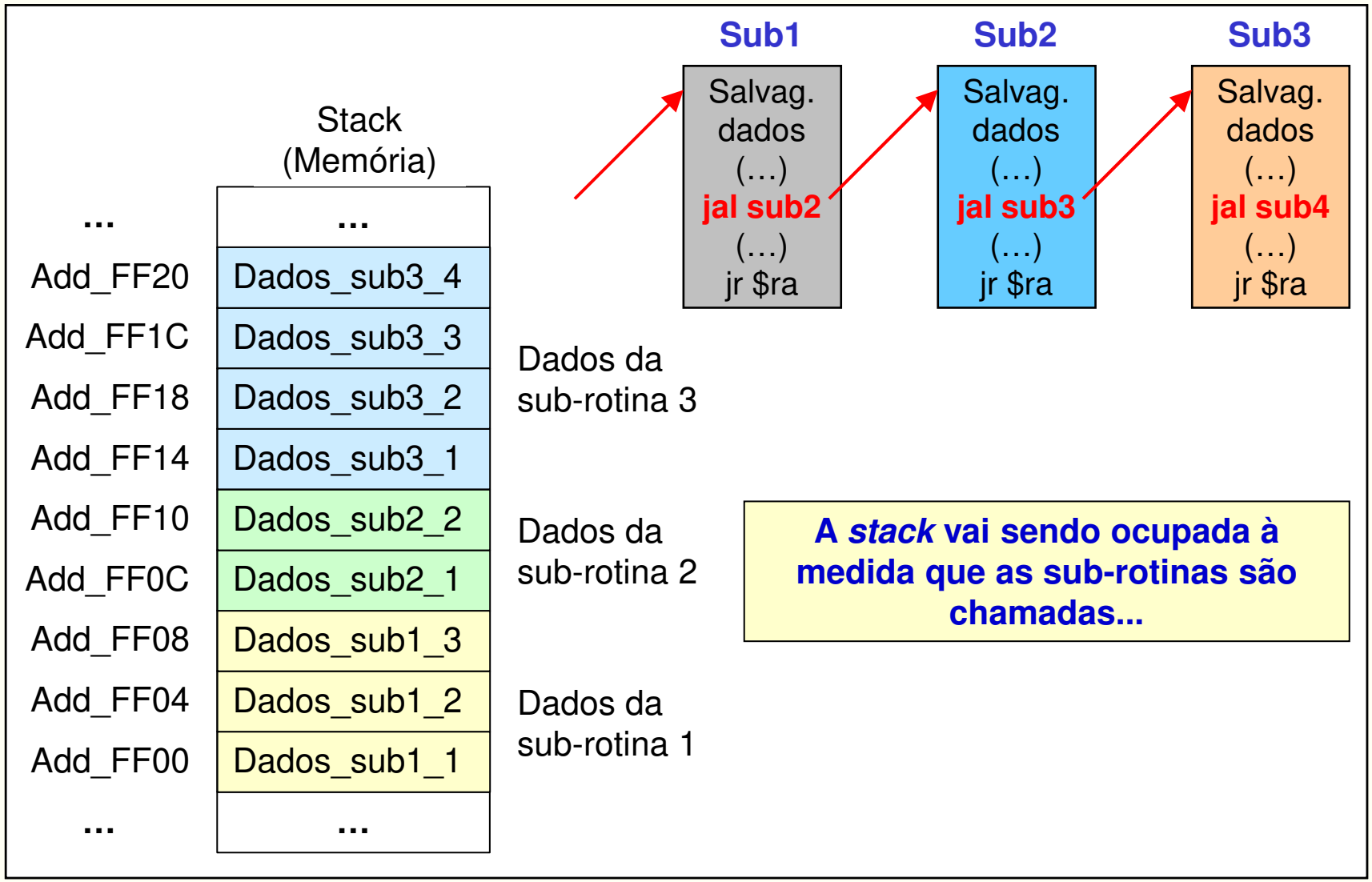
- **sub-rotina terminal** (sub-rotina folha, i.e., que não chama qualquer sub-rotina)
  - Só deve utilizar registos que não tem a responsabilidade de salvaguardar (`$t0..$t9`, `$v0..$v1` e `$a0..$a3`)
- **sub-rotina intermédia** (sub-rotina que chama outra(s) sub-rotina(s))
  - Deve utilizar os registos `$s0..$s7` para o armazenamento de valores que pretenda preservar durante a chamada à(s) sub-rotina(s) (i.e. registos cujo conteúdo venha a precisar mais tarde)
    - A utilização de qualquer um dos registos `$s0` a `$s7` implica a sua prévia salvaguarda na memória externa logo no início da sub-rotina e a respetiva reposição no final
  - Deve utilizar os registos `$t0..$t9`, `$v0..$v1` e `$a0..$a3` para os restantes valores

Armazenamento temporário de informação

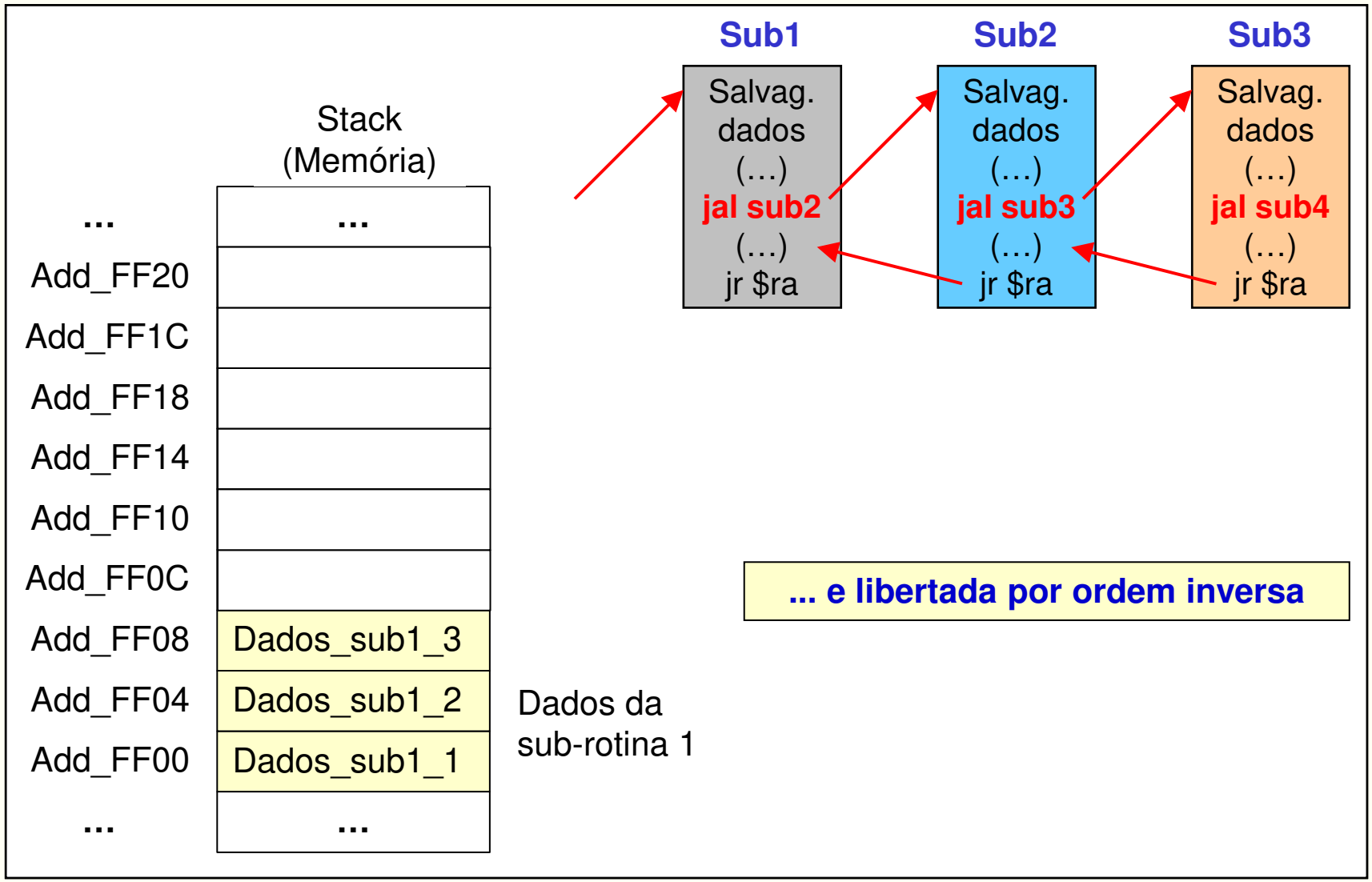


- Como poderemos garantir que cada sub-rotina, ao usar a memória para guardar os seus dados, não destrói os dados das restantes sub-rotinas?

Stack: espaço de armazenamento temporário



Stack: espaço de armazenamento temporário

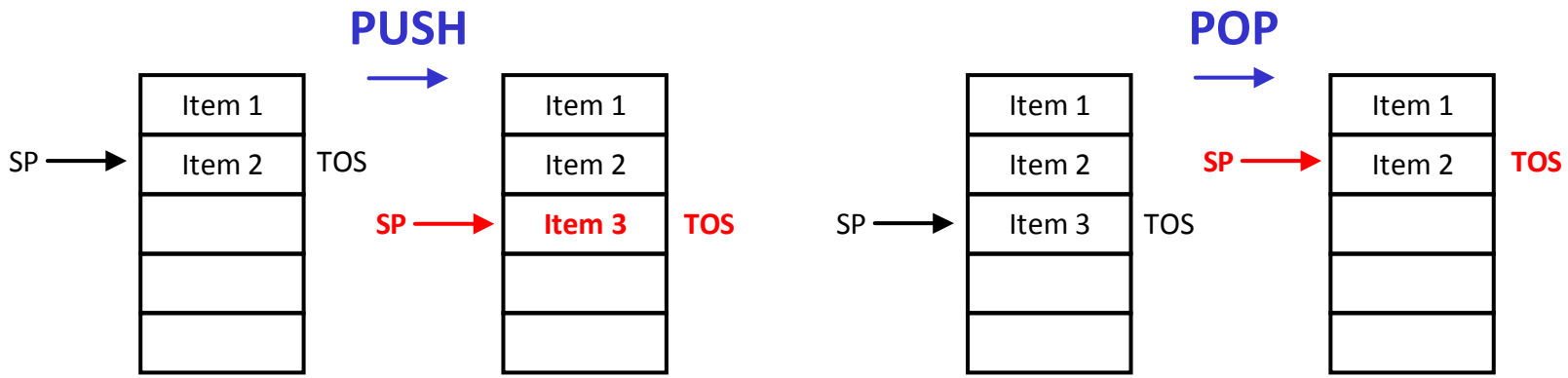


Stack: espaço de armazenamento temporário

- A estratégia de gestão dinâmica do espaço de memória - em que a última informação acrescentada é a primeira a ser retirada – é designada por **LIFO** (*Last In First Out*)
- A estrutura de dados correspondente é conhecida por ***stack*** ("pilha")
- As *stacks* são de tal forma importantes que muitas arquiteturas suportam diretamente instruções específicas para a sua manipulação (por exemplo a arquitetura Intel x86)
- A operação que permite acrescentar informação à *stack* é normalmente designada por **PUSH**, enquanto que a operação inversa é conhecida por **POP**

Stack: operações *push* e *pop*

- Estas operações têm associado um registo designado por ***Stack Pointer*** **(SP)**
- O registo ***Stack Pointer*** mantém, de forma permanente, **o endereço do topo da *stack*** (**TOS** - *top of stack*) e aponta sempre **para o último endereço ocupado**
  - Numa operação de **PUSH** é necessário pré-atualizar o *stack pointer* antes de copiar informação para a *stack*
  - Numa operação de **POP** é feita uma leitura da *stack*, do endereço atual do *stack pointer*, seguida de atualização do valor do *stack pointer*



Atualização do *stack pointer*

- A atualização do ***stack pointer***, numa operação de *push* (escrita de informação), pode seguir uma de duas estratégias:
  - Ser incrementado, fazendo crescer a *stack* no sentido crescente dos endereços
  - Ser decrementado, fazendo crescer a *stack* no sentido decrescente dos endereços

- **A estratégia de crescimento da *stack* no sentido dos endereços mais baixos é, geralmente, a adotada**

- A estratégia de crescimento da *stack* no sentido dos endereços mais baixos permite uma gestão simplificada da fronteira entre os segmentos de dados e de *stack*

Regras de utilização da *stack* na arquitetura MIPS

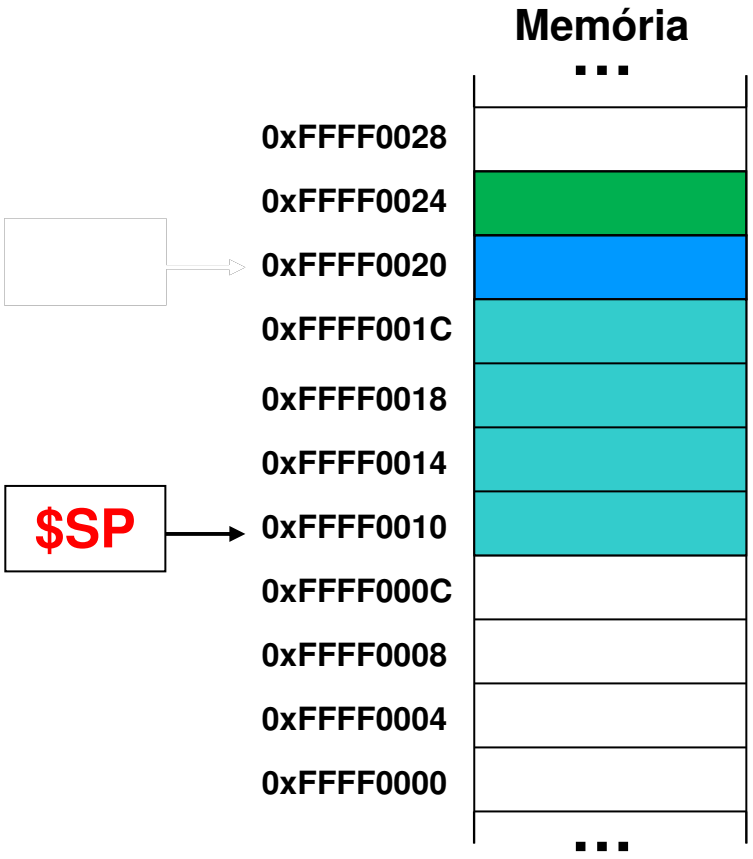
1. O registo **$sp** (*stack pointer*) contém o endereço da **última posição ocupada** da *stack*

$sp = $29

$SP: **0xFFFF0010**

2. A *stack* **cresce** no **sentido decrescente** dos endereços da memória

3. A *stack* **está organizada em words de 32 bits**



Regras de utilização da *stack* na arquitetura MIPS

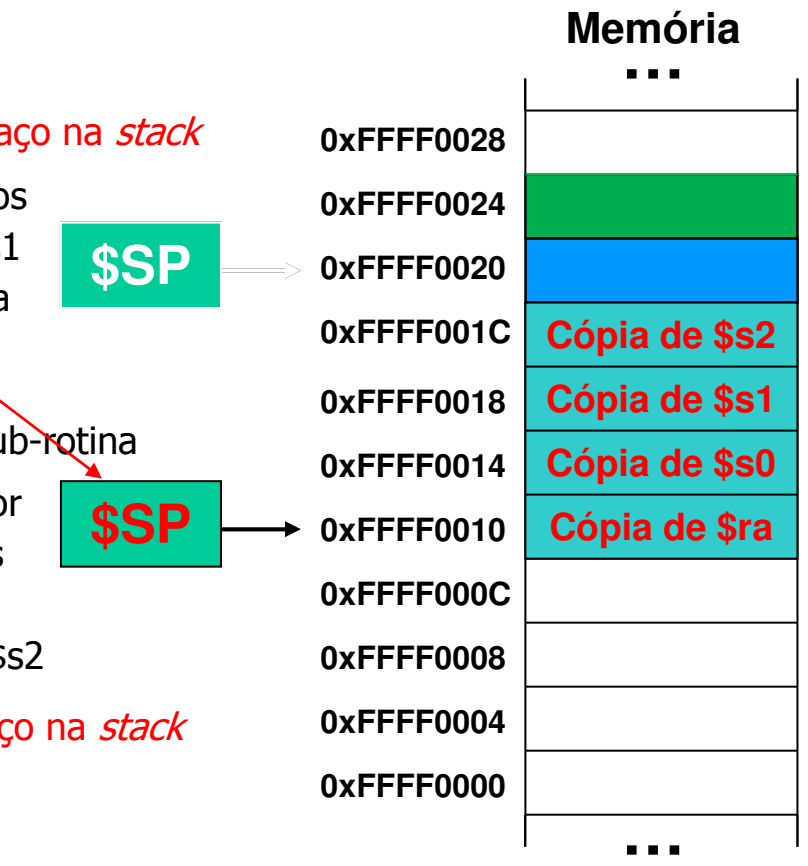
- Exemplo (salvaguarda de 4 registos)

```
sb1:  addiu  $sp,$sp,-16   # Reserva espaço na stack
      sw     $ra,0($sp)    # Copia registos
      sw     $s0,4($sp)    #  $ra, $s0, $s1
      sw     $s1,8($sp)    #  e $s2 para a
      sw     $s2,12($sp)   #  stack

        (...)              # Código da sub-rotina

      lw     $ra,0($sp)    # Repõe o valor
      lw     $s0,4($sp)    #  dos registos
      lw     $s1,8($sp)    #  $ra,
      lw     $s2,12($sp)   #  $s0, $s1 e $s2

      addiu  $sp,$sp,16    # Liberta espaço na stack
      jr     $ra           # Retorna
```



$SP: 0xFFFF0020 ⟶ 0xFFFF0010

Regras de utilização da *stack* na arquitetura MIPS

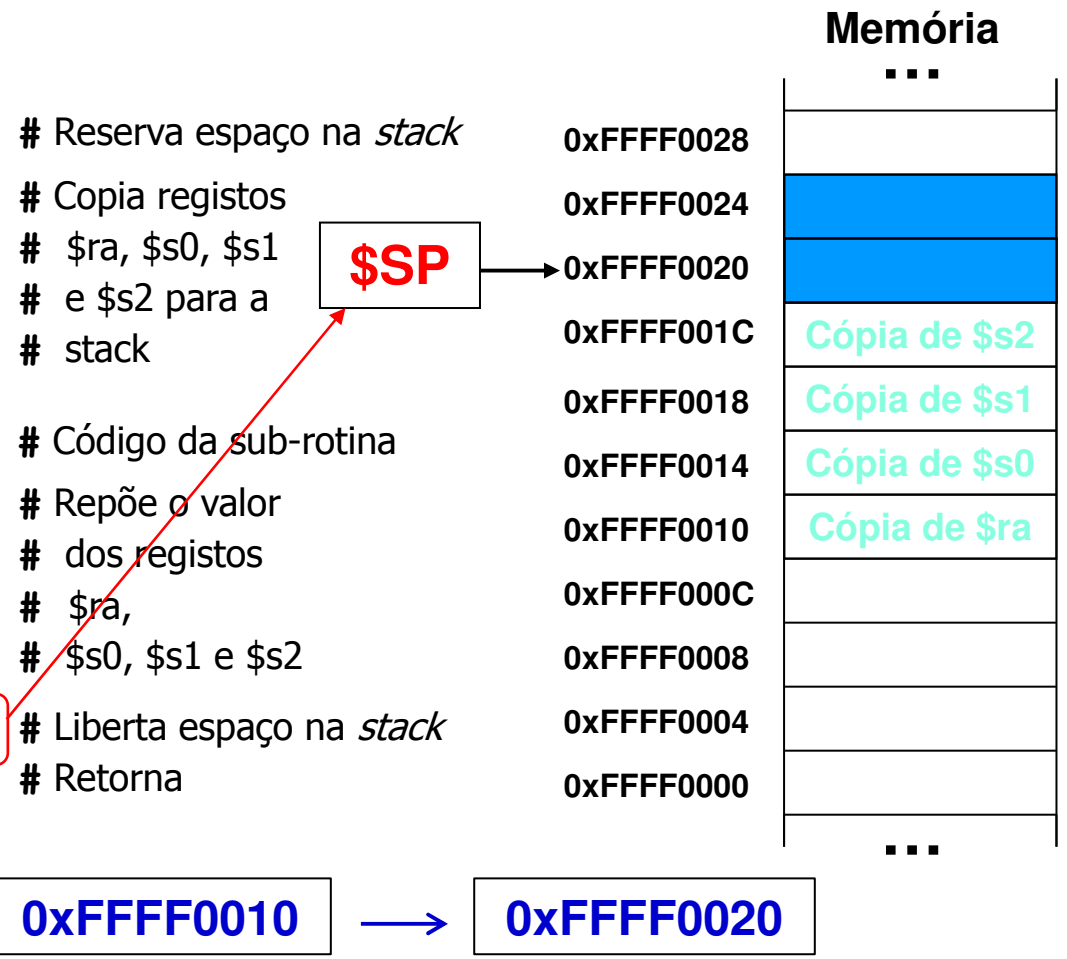
- Exemplo (salvaguarda de 4 registos)

```
sb1:  addiu  $sp,$sp,-16   # Reserva espaço na stack
      sw     $ra,0($sp)    # Copia registos
      sw     $s0,4($sp)    #  $ra, $s0, $s1
      sw     $s1,8($sp)    #  e $s2 para a
      sw     $s2,12($sp)   #  stack

         (...)             # Código da sub-rotina

      lw     $ra,0($sp)    # Repõe o valor
      lw     $s0,4($sp)    #  dos registos
      lw     $s1,8($sp)    #  $ra,
      lw     $s2,12($sp)   #  $s0, $s1 e $s2

      addiu  $sp,$sp,16    # Liberta espaço na stack
      jr     $ra           # Retorna
```



$SP: 0xFFFF0010 ⟶ 0xFFFF0020

Regras de utilização da *stack* na arquitetura MIPS

- Exemplo

